

COMPUTER ARCHITECTURE

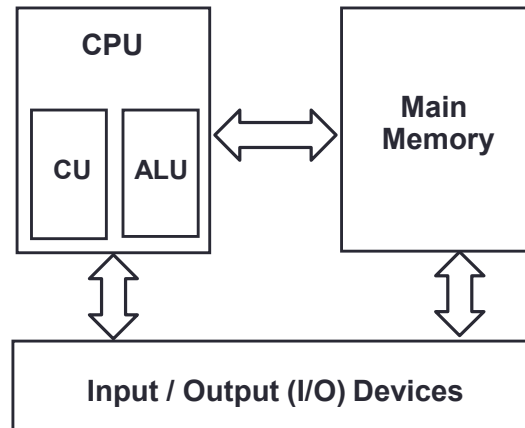


Hamid Sarbazi-Azad

Department of Computer Engineering
Sharif University of Technology,
Tehran, Iran

Computer System

In famous stored-program computer structure of John von Neumann (known as von Neumann model), CPU is connected to the memory system and I/O devices.



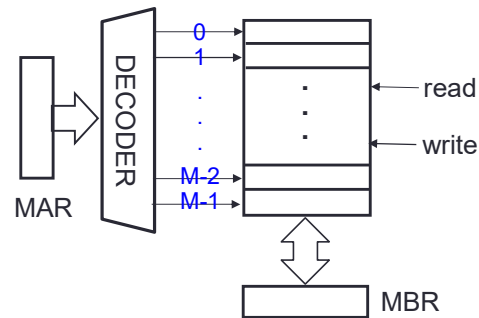
Main Memory

From the machine language programmer's view-point, **main memory** is an array of locations addressed from 0 to $M-1$ (M being the memory size).

All accesses to memory are realized via two registers: **MAR** (Memory Address Register) and **MBR** (Memory Buffer Register), and two control signals (**Read** and **Write**).

The address of the location to be read/written is placed in MAR.

The data communicated with memory is resided in MBR or MDR (Memory Data Register).



Main Memory

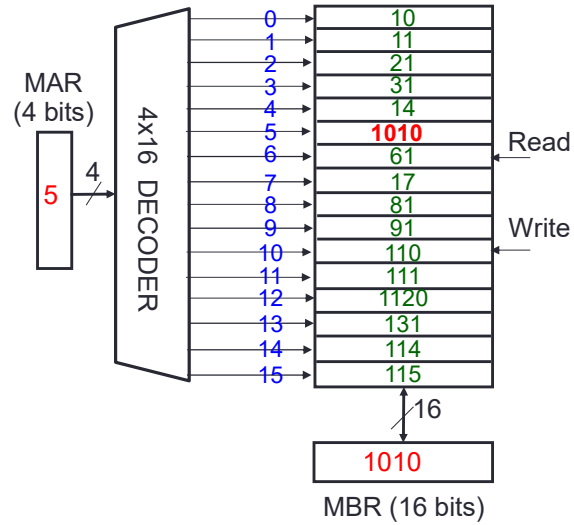
Example: In a 16x16 bit memory, show the steps of writing 1010 in address 5.

Write Operation:

Step 1. Store 5 in MAR.

Step 2. Store 1010 in MBR.

Step 2. Activate Write signal.
(data written in memory)



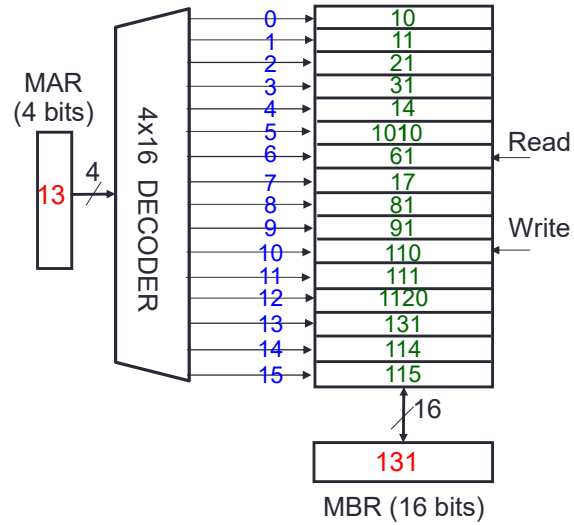
Main Memory

Example: In a 16x16 bit memory, show the steps of reading from address 13.

Read Operation:

Step 1. Store 13 in MAR.

Step 2. Activate Read signal.
(data is read into MBR)



Main Memory

Notation. We use M_{addr} to show a memory location with address $addr$. The content of a register or memory is shown by a parenthesis around it.

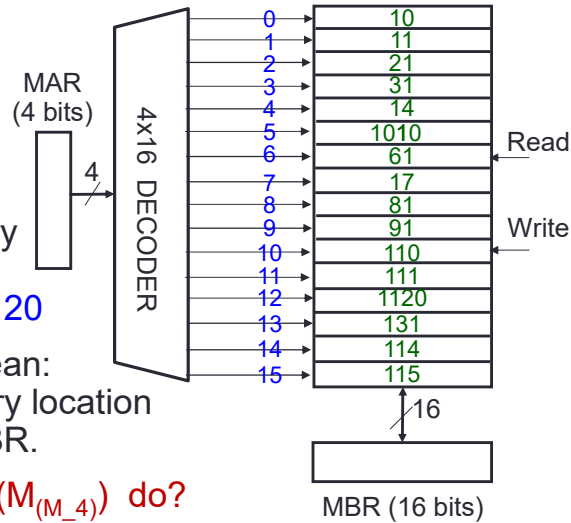
Example:

We have $(M_9) = 91$
and $(M_{(M_1)}) = 111$.

By $M_{10} \leftarrow (M_{12})$, we mean:
Copy the content of memory
location 12 into memory
location 10. $\rightarrow (M_{10}) = 1120$

By $MBR \leftarrow (M_{(MAR)})$, we mean:
Move the content of memory
addressed by MAR into MBR.

What does $M_{(M_0)} \leftarrow (M_0) + (M_{(M_4)})$ do?
 $\rightarrow (M_{10}) = 124$



Main Memory

Characteristics:

- **Addressable unit:** The smallest group of bits that has a unique address in memory. It is 8 bits in most machines.
- **Access width/length:** The number of bits that can be read or written in one memory access. It is different from machine to machine.
- **Word:** The number of bits of the operands that CPU process when a machine instruction is executed. It is n-bit in an n-bit machine. In first generation CPUs, we had n=8 or 16. For most current machines we have n=32. But machine with n=64 and larger word sizes are also available.

Typically, Addressable unit size < Word size < Access width.

If not explicitly mentioned, then assume all are of equal size.

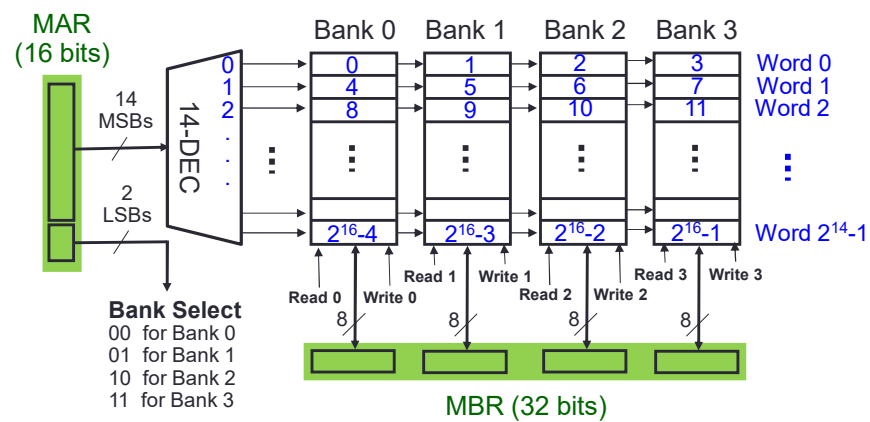
Sample Main Memory Architecture

Addressable unit = 8 bits

Word size = 32 bits

Access length = 32 bits

Memory capacity = 2^{16} addressable units = 2^{14} words



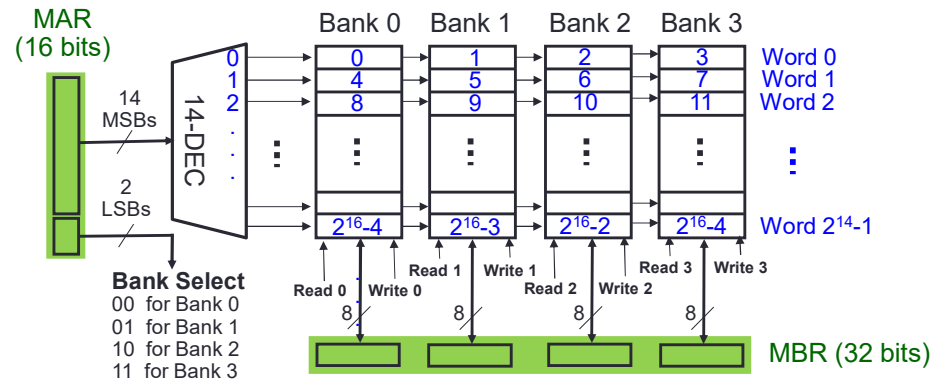
Word Alignment

Aligned Word Memory Architecture: Word address is a multiple of its length. In below memory @ 0, 4, 8, 12, ...

→ Word is read/written in one memory access 😊

Unaligned Word Memory Architecture: Word address can be anything. In below memory @ 0, 1, 2, 3, ...

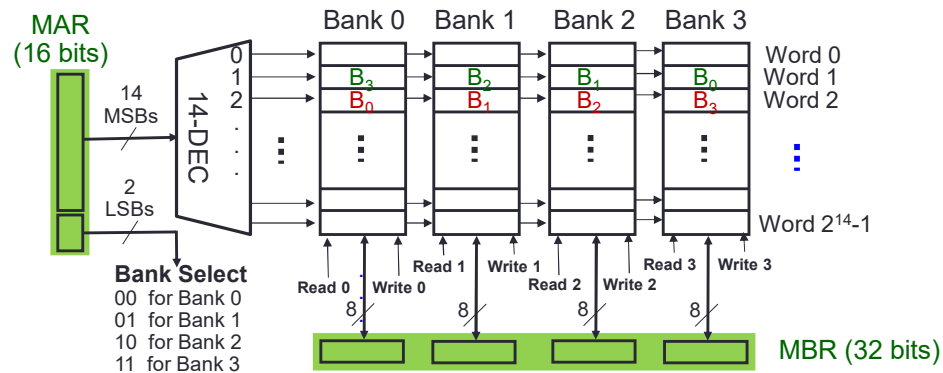
→ Word may be read/written in two memory accesses ☹



Endianness

Big Endian: When word is stored from most significant addressable unit toward least significant addressable units in consecutive memory addresses. Word $B_3B_2B_1B_0$ @ 0004h.

Little Endian: When word is stored from least significant addressable unit toward most significant addressable units in consecutive memory addresses. Word $B_3B_2B_1B_0$ @ 0008h.



Instruction Format

An Imperative Sentence in human languages is formatted as:

Subject + **verb** + **Objects**.

Example: **Mohsen** **call** **Mike**.

In machine language, we follow similar format. Note that in a computer the only Subject is CPU. So, we need not mention it.

So, a machine instruction looks like: **Operation** + **Objects**.

Operation is coded by a bit pattern named Operation Code (or OpCode, for short) and Objects are memory locations or registers addressed in binary patterns called Operands.

So, the instruction format in an n-address machine looks like:



Instruction Format

Example: In a 2-address machine, memory size is 128 units of 16 bits. Operations include **Add** (opcode=00), **Subtract** (opcode=01), **Move** (opcode=10), and **No-operation** (opcode=11).



For example, machine instruction **0100011110001010** means the operation is **Subtract** and the two operands are M_{15} and M_{10} . So, the instruction is: $M_{15} \leftarrow (M_{15}) - (M_{10})$.

As writing, reading and understanding programs of machine instructions is very difficult, we use symbolic representation of instructions, named assembly instruction. For example, the above machine instruction may be written as: **sub num1,num2** where **num1** and **num2** are symbolic equivalents of M_{15} and M_{10} and **sub** is symbolic equivalent (Mnemonic) of opcode=01.

Opcode

What information can Opcode give?

1. Operation type: **add, sub,**
2. Number of operands (if multiple has multiple instruction formats)
3. Instruction length (if machine has multiple instruction formats)
4. Type of operands: **integer, real, signed, ...**
5. Length of operands: **8 bits, 16 bits, n bytes**
6. Addressing mode of operands: **direct, indirect,...**

Addressing modes

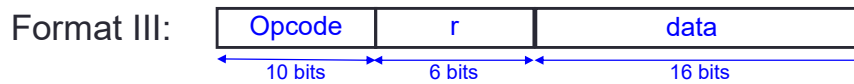
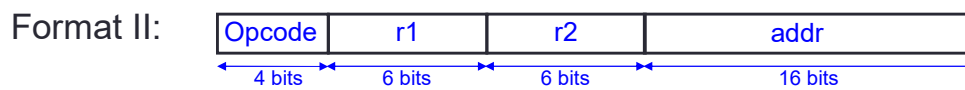
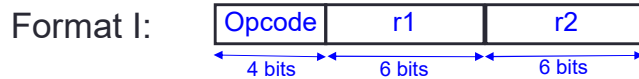
1. **Direct:** Address of operand is given in IF (instruction format)
 - Register direct (register number is given in IF)
 - Memory direct (memory address is given in IF)
2. **Indirect:** Address in IF indicated the register or location that contains the address of operand in memory
 - Register indirect
 - Memory indirect
3. **Immediate:** The operand is in IF (no register/memory access)
4. **Implied/Inherent:** Nothing in IF but CPU knows where is the operand!
5. **Indexed:** A direct address and a register number form the operand address. Operand's address is calculated by adding the content of given index register and direct address in IF.

Addressing modes (cont.)

6. **Relative:** The address is calculated by adding the address displacement given in IF and content of program counter (PC).
7. **Segment:** Final address (sometimes called physical address) is calculated by adding the address indicated in IF (sometimes called logical address) and the content of Segment register.
8. **Page:** Final address (sometimes called physical address) is calculated by concatenating the address indicated in IF (sometimes called logical address) and the content of Page register.

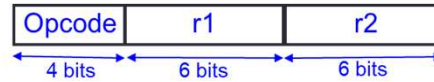
Sample 2-Address Machine

- Main memory size: 2^{16} addressable units (each 8 bits).
- Word size: 16 bits, unaligned, big endian
- 64 general purpose registers: R0, R1, ..., R63
- Numbers are represented in 2's-complement
- Addressing modes: Direct, Immediate, Indexed
- 3 instruction formats



Sample 2-Address Machine

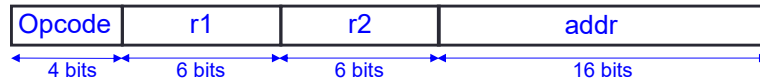
Instruction in Format I:



Opcode	Mnemonic	Operation
-----	-----	-----
0000	mov r1,r2	$r1 \leftarrow (r2);$
0001	add r1,r2	$r1 \leftarrow (r1) + (r2);$
0010	sub r1,r2	$r1 \leftarrow (r1) - (r2);$
0011	and r1,r2	$r1 \leftarrow (r1) \wedge (r2);$
0100	or r1,r2	$r1 \leftarrow (r1) \vee (r2);$
0101	xor r1,r2	$r1 \leftarrow (r1) \text{ xor } (r2);$

Sample 2-Address Machine

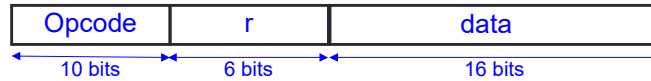
Instruction in Format II:



Opcode	Mnemonic	Operation
0110	mov r1,addr(r2)	$r1 \leftarrow (M_{addr+(r2)});$
0111	mov addr(r2),r1	$M_{addr+(r2)} \leftarrow (r1);$
1000	mov r1,r2,addr	$r1 \leftarrow (r2); r2 \leftarrow (M_{addr});$
1001	mov addr,r1,r2	$M_{addr} \leftarrow (r1); r1 \leftarrow (r2);$
1010	je r1,r2,addr	if $(r1)=(r2)$ then $PC \leftarrow addr;$
1011	jne r1,r2,addr	if $(r1) \neq (r2)$ then $PC \leftarrow addr;$
1100	jl r1,r2,addr	if $(r1) < (r2)$ then $PC \leftarrow addr;$
1101	jh r1,r2,addr	if $(r1) > (r2)$ then $PC \leftarrow addr;$
1110	call r1,addr(r2)	$r1 \leftarrow (PC); PC \leftarrow addr+(r2);$

Sample 2-Address Machine

Instruction in Format III:



Opcode	Mnemonic	Operation
1111000000	mov r,#data	$r \leftarrow \text{data};$
1111000001	add r,#data	$r \leftarrow (r) + \text{data};$
1111000010	sub r,#data	$r \leftarrow (r) - \text{data};$
1111000011	and r,#data	$r \leftarrow (r) \wedge \text{data};$
1111000100	or r,#data	$r \leftarrow (r) \vee \text{data};$
1111000101	xor r,#data	$r \leftarrow (r) \text{ xor } \text{data};$

Sample 2-Address Machine

1) Determine the length of machine registers.

Answer:

$$\begin{array}{lll} L_{\text{MAR}} = 16 \text{ bits} & L_{\text{MBR}} = 16 \text{ bits} & L_{\text{PC}} = 16 \text{ bits} \\ L_{\text{IR}} = 32 \text{ bits} & L_{\text{R0..R63}} = 16 \text{ bits} & \end{array}$$

2) Write a program to summate elements of an array of 100 elements. Translate your assembly program into machine code.

3) Write a program to accumulate the content of R0, R1, ..., R62 into R62.

Sample 2-Address Machine

2) Write a program to summate elements of an array of 100 elements. Translate your assembly program into machine code.

```
org    100h

xor     R0,R0      ; accumulator
mov     R1,R0      ; index register
mov     R2,#200    ; last element's index for loop control

loop:   mov     R3,A(R1) ; loading the i-th element of A
        add     R0,R3   ; adding it to accumulator
        add     R1,#2   ; advancing the index register for next element
        jne     R1,R2,loop ; if index is not 200 then continue

        mov     sum,R0,R0 ; store the result into variable sum

        call    R0,0(R63) ; return to OS. Assume R63 has return address

sum:    dw      0       ; variable to keep summation
A:      dw      100 dup(?) ; definition of a 100-element array
        end          ; end of program
```

(c) Hamid Sarbazi-Azad

Computer Architecture – Lecture #4: Overview of Comp. Structure & Machine Language

22

Sample 2-Address Machine

2) Write a program to summate elements of an array of 100 elements. Translate your assembly program into machine code.

Address	Code (H)	Assembly Line	
0100		org 100h	
0100	5000	xor R0,R0	0101 000000 000000
0102	0040	mov R1,R0	0000 000001 000000
0104	F00200C8	mov R2,#200	1111000000 000010 [00C8h]
0108	60C10120	loop: mov R3,A(R1)	0110 000011 000001 [0120h]
010C	1003	add R0,R3	0001 000000 000011
010E	F0410002	add R1,#2	1111000001 000001 [0002h]
0112	B0420108	jne R1,R2,loop	1011 000001 000010 [0108h]
0116	9000011E	mov sum,R0,R0	1001 000000 000000 [011Eh]
011A	E03F0000	call R0,0(R63)	1110 000000 111111 [0000h]
011E	0000	sum: dw 0	
0120	????????	A: dw 100 dup(?)	
		end	

Opcode

r1

r2

4 bits

6 bits

6 bits

Opcode

r1

r2

addr

4 bits

6 bits

6 bits

16 bits

Opcode

r

data

10 bits

6 bits

16 bits

Sample 2-Address Machine

3) Write a program to accumulate the content of R0, R1, R2, ..., R62 into R62.

```
org    0

add    R62,R61
mov    R61,#61

loop:  add    R62,R60

      mov    R60,R60,loop
      sub    R60, #1
      mov    loop,R60,R60

      xor    R60,R60
      sub    R61, #1

      jne    R60,R61,loop

      call   R0,0(R63)
      end
```

Homework: Generate the machine code of the above assembly program.

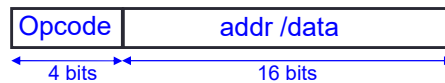
One-address Machine

One operand is kept in a special purpose register named ACC (Accumulator) and addressed inherently.

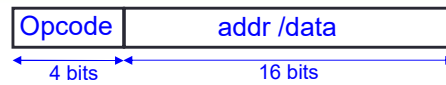
Example: In a one-address machine:

- main memory size = 2^{16} addressable units (each 4 bits)
- word size is 16 bits (unaligned, big endian)
- 2's-complement numbers
- Implied, direct, indirect, and immediate addressing modes

Instructions are coded in one format.



One-Address Machine



Opcode	Mnemonic	Operation
0000	load #data	$ACC \leftarrow data;$
0001	add #data	$ACC \leftarrow (ACC) + data;$
0010	load addr	$ACC \leftarrow (M_{addr});$
0011	store addr	$M_{addr} \leftarrow (ACC);$
0100	add addr	$ACC \leftarrow (ACC) + (M_{addr});$
0101	sub addr	$ACC \leftarrow (ACC) - (M_{addr});$
0110	and addr	$ACC \leftarrow (ACC) \wedge (M_{addr});$
0111	or addr	$ACC \leftarrow (ACC) \vee (M_{addr});$
1000	xor addr	$ACC \leftarrow (ACC) \text{ xor } (M_{addr});$
1001	load (addr)	$ACC \leftarrow (M_{(M_{addr})});$
1010	store (addr)	$M_{(M_{addr})} \leftarrow (ACC);$
1011	j addr	$PC \leftarrow addr;$
1100	jz addr	if $(ACC) = 0$ then $PC \leftarrow addr;$
1101	jnz addr	if $(ACC) \neq 0$ then $PC \leftarrow addr;$
1110	jn addr	if $(ACC) < 0$ then $PC \leftarrow addr;$
1111	jp addr	if $(ACC) > 0$ then $PC \leftarrow addr;$

One-address Machine

Write a program to generate first 20 elements of Fibonacci series and store them in an array.

```

        org    0

loop:   load    (pointer_1)           load    counter
        store   sum                  add     #-1
        load    (pointer_2)           store   counter
        add     sum
        store   (pointer_3)           jnz     loop

        load    pointer_1             j       OS_return_addr
        add     #4
        store   pointer_1
        add     #4
        store   pointer_2
        add     #4
        store   pointer_3

        pointer_1: dw    array
        pointer_2: dw    array+4
        pointer_3: dw    array+8
        counter:  dw    18
        sum:      dw    ?
        array:    dw    0, 1, 18 dup(?)

        end

```

Homework: Generate the machine code of the above assembly program.

Zero-address Machine (Stack Machine)

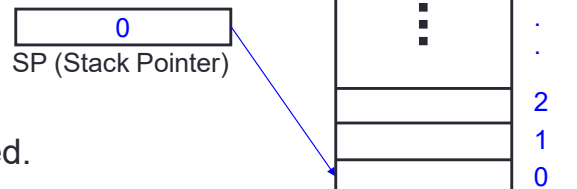
All operands are kept in a hardware stack inside CPU.

Hardware Stack: an internal memory of S words that follows LIFO (last in first out) strategy for writing (push) and reading (pop) data.

SP (Stack Pointer) register points to top of stack where it is ready to get new data.

When data is pushed on top of stack, SP is incremented.

For pop, the SP is decremented, the word addressed by SP is returned.



Zero-address Machine (Stack Machine)

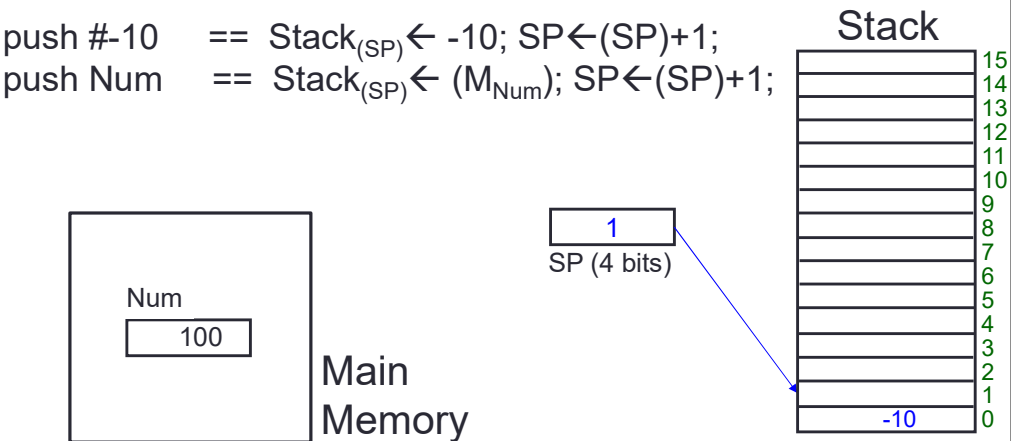
Example: Suppose a hardware stack of 16 words. SP is reset to 0 indicating that stack is empty. Perform the following push and pop operations. Num is a variable in memory initialize by 100.

push #-10 == $\text{Stack}_{(\text{SP})} \leftarrow -10; \text{SP} \leftarrow (\text{SP})+1;$



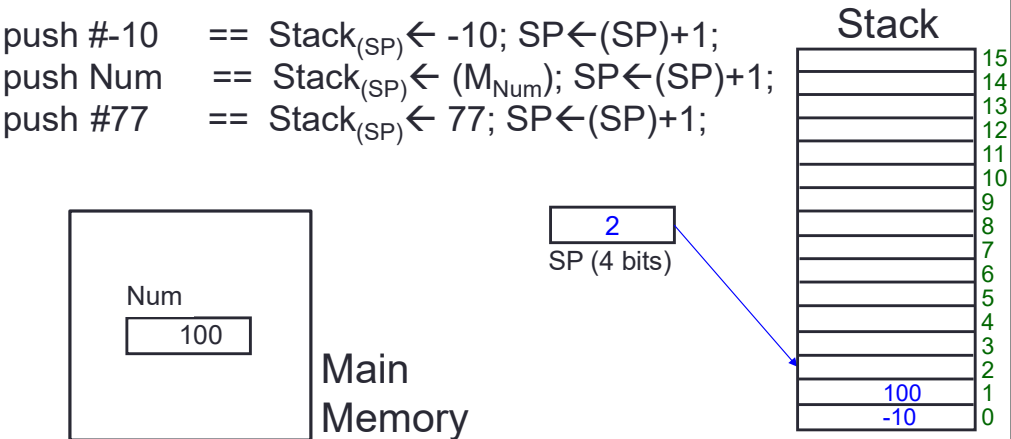
Zero-address Machine (Stack Machine)

Example: Suppose a hardware stack of 16 words. SP is reset to 0 indicating that stack is empty. Perform the following push and pop operations. Num is a variable in memory initialize by 100.



Zero-address Machine (Stack Machine)

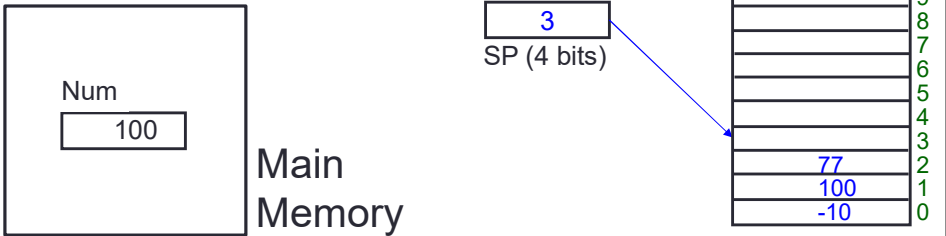
Example: Suppose a hardware stack of 16 words. SP is reset to 0 indicating that stack is empty. Perform the following push and pop operations. Num is a variable in memory initialize by 100.



Zero-address Machine (Stack Machine)

Example: Suppose a hardware stack of 16 words. SP is reset to 0 indicating that stack is empty. Perform the following push and pop operations. Num is a variable in memory initialize by 100.

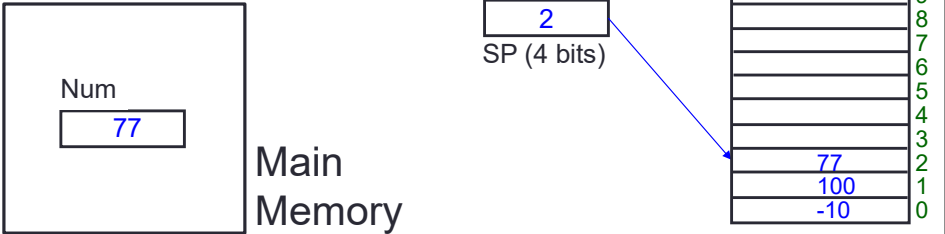
```
push #-10 == Stack(SP) ← -10; SP ← (SP)+1;  
push Num  == Stack(SP) ← (MNum); SP ← (SP)+1;  
push #77  == Stack(SP) ← 77; SP ← (SP)+1;  
pop  Num  == SP ← (SP)-1; Mnum ← Stack(SP);
```



Zero-address Machine (Stack Machine)

Example: Suppose a hardware stack of 16 words. SP is reset to 0 indicating that stack is empty. Perform the following push and pop operations. Num is a variable in memory initialize by 100.

```
push #-10 == Stack(SP) ← -10; SP ← (SP)+1;  
push Num  == Stack(SP) ← (MNum); SP ← (SP)+1;  
push #77  == Stack(SP) ← 77; SP ← (SP)+1;  
pop  Num  == SP ← (SP)-1; Mnum ← Stack(SP);
```

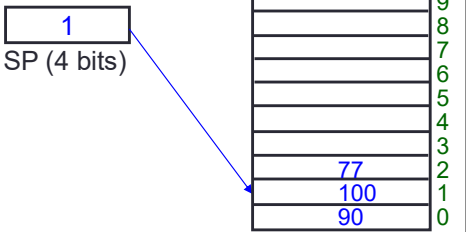


Zero-address Machine (Stack Machine)

To execute a function say **func(a,b)**, the operands **a** and **b** are taken from top of stack and the result **func(a,b)** is pushed into stack.

```
add == pop temp1;  
      pop temp2;  
      temp3 ← (temp1) + (temp2);  
      push temp3
```

Example: add



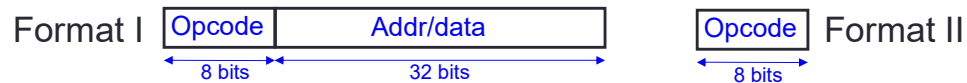
Zero-address Machine (Stack Machine)

In a stack machine, all instructions (except push and pop) have no operand fields in the instruction format. So, **almost** all instructions have no operand fields in the instruction format.

Example: In a stack machine:

- main memory size = 2^{32} addressable units (each 8 bits)
- word size is 32 bits
- 2's-complement numbers
- Implied, direct, immediate addressing modes

Instructions are coded in two formats.



Stack Machine

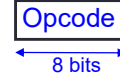
Format I

Opcode	Mnemonic	Operation
00000000	push #data	$\text{Stack}_{(\text{SP})} \leftarrow \text{data}; \text{SP} \leftarrow (\text{SP})+1;$
00000001	push addr	$\text{Stack}_{(\text{SP})} \leftarrow (\text{M}_{\text{addr}}); \text{SP} \leftarrow (\text{SP})+1;$
00000010	pop addr	$\text{SP} \leftarrow (\text{SP})-1; \text{M}_{\text{addr}} \leftarrow (\text{Stack}_{(\text{SP})});$



Format II

Opcode	Mnemonic	Operation
00000011	add	$\text{Stack}_{(\text{SP})-2} \leftarrow (\text{Stack}_{(\text{SP})-2}) + (\text{Stack}_{(\text{SP})-1}); \text{SP} \leftarrow (\text{SP})-1;$
00000100	sub	$\text{Stack}_{(\text{SP})-2} \leftarrow (\text{Stack}_{(\text{SP})-2}) - (\text{Stack}_{(\text{SP})-1}); \text{SP} \leftarrow (\text{SP})-1;$
00000101	swap	$\text{Stack}_{(\text{SP})-1} \leftarrow (\text{Stack}_{(\text{SP})-2})$ and $\text{Stack}_{(\text{SP})-2} \leftarrow (\text{Stack}_{(\text{SP})-1});$
00000110	sp++	$\text{SP} \leftarrow (\text{SP})+1;$
00000111	sp--	$\text{SP} \leftarrow (\text{SP})-1;$
00001000	j	$\text{PC} \leftarrow (\text{Stack}_{(\text{SP})-1}); \text{SP} \leftarrow (\text{SP})-1;$
00001001	jz	if $(\text{Stack}_{(\text{SP})-2}) = 0$ then $\text{PC} \leftarrow (\text{Stack}_{(\text{SP})-1}); \text{SP} \leftarrow (\text{SP})-2;$
00001010	jnz	if $(\text{Stack}_{(\text{SP})-2}) \neq 0$ then $\text{PC} \leftarrow (\text{Stack}_{(\text{SP})-1}); \text{SP} \leftarrow (\text{SP})-2;$
00001000	jn	if $(\text{Stack}_{(\text{SP})-2}) < 0$ then $\text{PC} \leftarrow (\text{Stack}_{(\text{SP})-1}); \text{SP} \leftarrow (\text{SP})-2;$



Stack Machine

Write a program to generate the sum of first 20 elements of Fibonacci series and store it in variable **sum**.

```

    org    0
loop: push  x          push  counter
      push  y          push  #-1
      add                   add
      pop    x          pop    counter
      sp++          sp++
      push  sum          push  #loop
      add                   jnz
      pop    sum          push  #OS_return_addr
      push  x          j
      push  y          sum:    dw    1
      add          counter:  dw    10
      pop    y          x:     dw    0
      sp++          y:       dw    1
      push  sum          end
      add
      pop    sum

```

Homework: Generate the machine code of the above assembly program. Can you write a better program?

END OF SLIDES

QUESTIONS?